Keyboard Layout Optimization

Gowtham S, EE24B017

September 2025

Introduction

This report describes how simulated annealing was used to optimize the keyboard layout for a given piece of text.

Functions used :

`preprocess_text()`

This function converts the input text to lowercase and replaces every character not in the allowed set with a space. It ensures that only the supported characters remain, which makes the cost calculation consistent and reliable.

`path_length_cost()`

This function computes the total typing cost for a given text and keyboard layout. It does so by summing the Euclidean distances between the positions of every pair of consecutive characters in the text.

`Euclid_dist()`

This function calculates the straight-line (Euclidean) distance between two points on the keyboard grid using the formula $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. It is used by `path_length_cost` to measure finger travel distance between key presses.

`get_neighbour()`

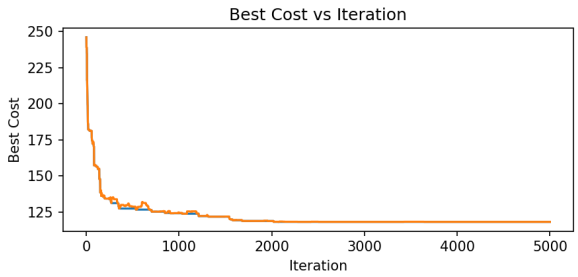
This function generates a neighboring layout by randomly swapping the positions of two characters in the current layout. Such small changes allow simulated annealing to explore different layouts in search of an optimal one.

`simulated_annealing()`

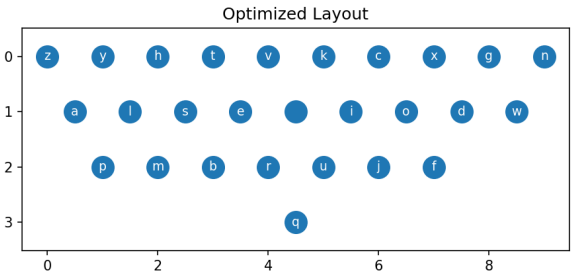
This function performs the optimization to minimize the total path-length cost. Starting from an initial layout, it repeatedly explores neighboring layouts and accepts them if they improve the cost or, occasionally, even if they worsen it—based on a temperature-controlled probability. This approach helps escape local minima and converge to a near-optimal layout.

`sweep_sa()`

The `sweep_sa` function automates parameter studies for Simulated Annealing. It sweeps over either the initial temperature (t_0) or the number of iterations while keeping the other fixed. For each value, it runs the optimizer on the input text, records the final cost and runtime, and generates a plot of Final Optimized Cost versus the chosen parameter. This helps visualize how t_0 or iterations affect convergence.



(a) Plot of the loss function vs iterations



(b) Final layout of the keyboard

Figure 1: Plot of cost function and the final layout on the starter data

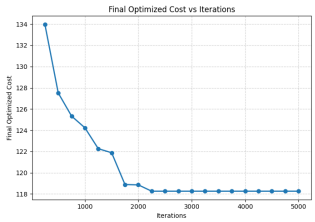
Effect of Iterations and Initial Temperature on Optimization

I ran simulated annealing on two texts—a short demo and a longer passage(present in text.txt)—varying initial temperature t_0 and number of iterations to study their impact on convergence quality and runtime.

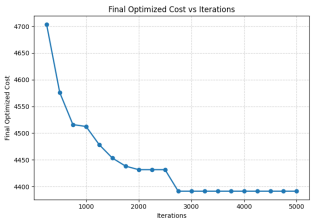
Short text: - Baseline cost: 246.06 - *Iterations:* Increasing from 250 to 5000 reduced final cost from 133.98 to 118.26, with most gains by 2250 iterations; runtime increased modestly from 0.01s to 0.20s. - *Initial temperature:* High t_0 (0.8–1.0) gave lowest costs (118.26), while lower t_0 (< 0.5) slightly raised costs ($\approx$ 122–124).

Long text: - Baseline cost: 7464.26 - *Iterations:* More iterations improved cost from 4703.67 (250 iterations) to 4391.36 ($>$ 2750 iterations); runtime rose from 0.28s to 8.15s. - *Initial temperature:* Higher t_0 (> 0.7) produced optimal costs (≈ 4391.36), while lower t_0 slightly increased costs (≈ 4423.82).

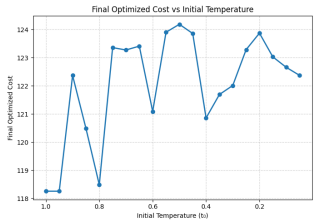
Overall: Moderate-to-high initial temperatures (t_0) and sufficient iterations ensure effective convergence, allowing the algorithm to explore the layout space thoroughly and avoid getting trapped in local minima. Runtime scaled almost linearly with both text length and iteration count—short text runs stayed under 0.2 s even at 5000 iterations, while longer text required nearly 8 s at the same settings. This highlights that runtime overhead is negligible for small inputs but grows noticeably for larger texts as more swaps are evaluated per iteration. Improvements in cost plateau once the layout space is well explored, indicating diminishing returns from further computation. Runtime was almost a constant for different values of initial temperature, signifying almost no correlation between the two.



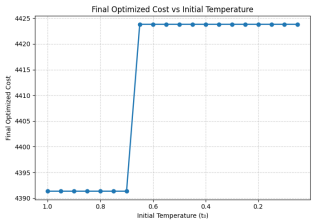
Iterations - Short text



Iterations - Long text



t_0 - Short text



t_0 - Long text

Running the Code and Changing Parameters

Run the code on the starter data with:

```
python3 kbd_optim.py
```

Default settings: `iters=5000`, `alpha=0.999`, `t0=1.0`, `epochs=1`, seed = 0.
To customize:

- Change the input file path at **line 359** (`filename` variable).
- Update the random seed at **line 330** in `random.Random()`.
- Modify SA parameters at **line 344** in the `params` object.

Save and rerun to see how the new settings affect results.